T Pradeep
192411247

1) Regular Expression Design

A Regular Expression is used to represent patterns in strings it helps identify whether an input follows a particular format.

1) Binary string that end with 01

$$RE = (0+1)^*01$$

The string can contain any combination of 0 and 1, but it must end with 01. Ex: 01, 101, 1101

2) Binary string containing 11 or 00

$$RE = (0+1)^*(11+00)(0+1)^*$$

The string must contain either 11 or 00 at least once. Ex: 110, 001, 1001

3) string containing at least one occurrence of ab

$$RE = (a+b)^*ab(a+b)^*$$

The string must contain the substring ab at least once Ex:- ab, ab, bab

4) string having an even number of zeros

$$R.E = 1^*(01^*01^*)^*$$

The string must contain an even number of zeros zeros occurs in pairs Ex: 00, 1010, 11, 0011

) Task 2 :- Regular Expression Analysis

The regular expression is missing in the uploaded requstion Paper. Assuming the expression

$$R = (0+1)^*01$$

1.) language represented

It represent all binary string that end wind substring 01

2.) Alphabet and substring

* Alphabet: 1.
* Substring: 01

3.) Five accepted string

10, 00, 11, 11

These string do not end with 01

4.) kleene start and union

* kleene start(*): Allows zero or more repetitions of symbols
* union(+): Represent a choice b/w two symbols. Here, either 0 or 1 can occur.

Conclusion:- Regular expression are useful for validationg input Patterns and identify valid and validstring.

CO3- Assc

T. Pradeep
192411247
Slot-B

1) Context free Grammar Construction

A context-free Grammar (CFG) is used to describe the structure of a language it consists of terminals, not-terminal, a start symbol and Production rules.

1.) Equal number of a's followed by b's

$S \rightarrow aSb|\epsilon$

* start symbol: S * Terminals

* Non-terminal: S Ex: ab, aabb, aaabbb.

2.) Palindrome over 1

$S \rightarrow 0S0|1S1|0|1|\epsilon$

*start symbol: S *Terminals: 1

* Non-~~terminals~~: S Ex:- 00, 11, 101, 1001

3) Arthmethic expression involving addition.

$E \rightarrow E+E|id$

* start symbol: E .Terminals: Non-terminal

Ex: a+b, x+y.

4.) Balanced Parentheses

$S \rightarrow (S)S|\epsilon$

* start symbal: S * Terminals * Non terminal S.

Ex: () (())()()

4) Derivation and Parse Tree

Assumming the intended grammar is

$S \rightarrow aSb | \epsilon$

1.) Components of grammar

n	Generated string
0	ε
1	ab
2	aabb
3	aaabbb

2.) Generate string for n=0,1,2,3

+ start symbol: S Terminals: Non-terminals

+ Production rules: $S \rightarrow$ ~~x~~$aSb|\epsilon$

3.) leftmost derivation of aaabb

In leftmost derivation the left most non terminals is replaced first.

$S \Rightarrow aSb$

$\Rightarrow aaSbb$

$\Rightarrow aaabbb$

$\Rightarrow aaabbb$

4.) Righmost derivation of aabbb.

In righmost derivation the righmost nonterminal is replaced first

$S \Rightarrow aSb$

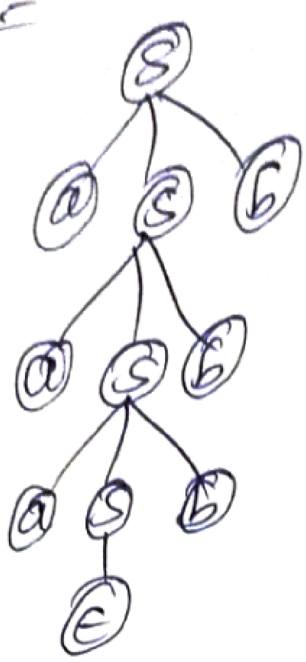
$\Rightarrow aaSbb$

$\Rightarrow aaaSbbb$

$\Rightarrow aaabbb$

√ CO3- Ascr

Parse tree:-



5·) Task 5 - Grammar Design for Arithmethic Expssion

A Programming Platform using arthmethic expsions containing id entifies addition, multiplication and Parenth eses

1·) Production rules

$E \longrightarrow E+T/T$ $T \longrightarrow T*F/F$ $F \longrightarrow (E)/id$

2·) Grammar componets

* Start symbols: E

* Terminals.

46/50